

Real-Time Object Detection — Project Report

CPU-only webcam object detection with YOLOX-Tiny and ONNX Runtime

Pratik Jakanur · github.com/Pratik7595/real-time-object-detection

Requirement	Target	Achieved
Frame rate	> 15 FPS	38.0 FPS (2.5x the requirement)
Object classes	at least 10	80 (full COCO set)
Demo video	30–60 seconds	43 seconds
Hardware	platform-independent	CPU-only; no GPU used anywhere

1. Implementation Approach

Model selection

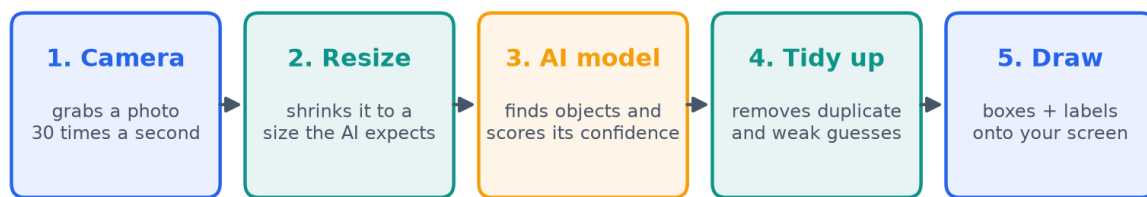
Four candidates were compared on accuracy, licence, dependency weight and ARM compatibility before any code was written:

Model	Published mAP	Licence	Runtime dependency
YOLOX-Tiny (chosen)	32.8	Apache-2.0	onnxruntime, ~16 MB
YOLOv8n	37.3	AGPL-3.0	ultralytics + PyTorch, ~250 MB
YOLOv5n	28.0	AGPL-3.0	PyTorch
MobileNet-SSD v2	~22	Apache-2.0	OpenCV DNN only

- **Licence was decisive.** YOLOv8n and YOLOv5n are AGPL-3.0 — strong copyleft, not permissive. Using them would arguably place the entire repository under AGPL, so both were ruled out on licensing grounds despite YOLOv8n's higher accuracy.
- **No deep-learning framework.** Megvii publish a pre-exported ONNX file, so there is no PyTorch anywhere in the project. Install size is ~40 MB rather than ~300 MB.
- **Best accuracy per millisecond.** 32.8 mAP at a 416px input beats YOLOv5n's 28.0 at 640px, while MobileNet-SSD at ~22 mAP missed small objects too visibly for a live demo.

Runtime and architecture

- **Runtime:** ONNX Runtime on the CPU execution provider. Pure pip wheels for x86_64 and ARM64, no CUDA toolchain, identical install command on every platform.
- **Custom post-processing:** ONNX Runtime returns raw tensors, so letterboxing, grid decoding and non-maximum suppression are implemented directly in NumPy (~120 lines) rather than being handled by a framework.
- **Two threads, not three:** A capture thread keeps only the newest frame and discards stale ones. With a queue, a slow detector makes the displayed image drift further behind reality every second; dropping frames bounds latency instead. Measured capture cost while the camera outran the detector: 0.11 ms.



This whole loop takes about 26 milliseconds — roughly 38 times every second.

The five stages every frame passes through.

Optimisations applied

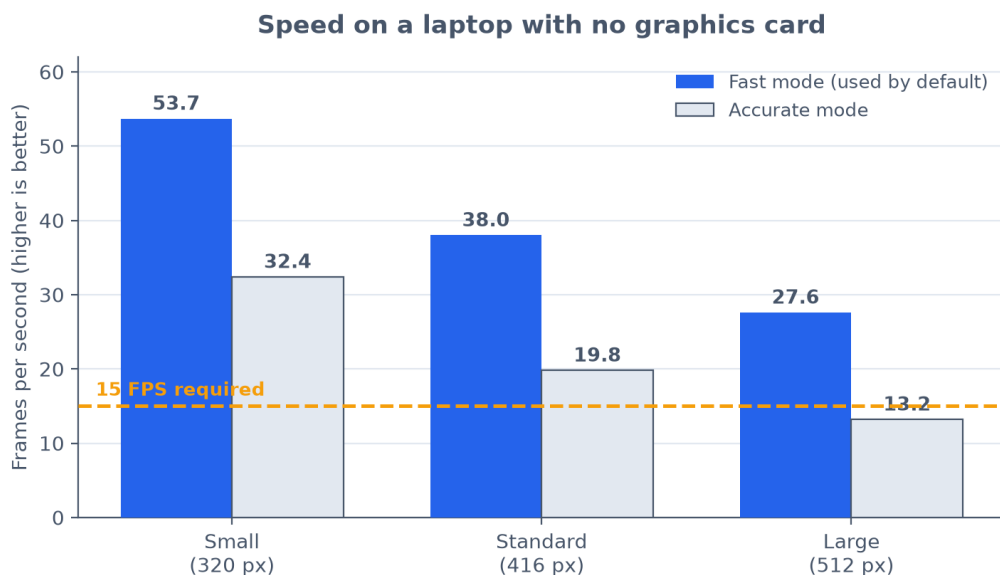
Technique	Effect	Kept?
INT8 static quantisation	1.9x faster, model 20.2 MB to 5.2 MB	Yes — now the default
Preallocated preprocessing buffers	Preprocess 3.53 ms to 1.04 ms (-71%)	Yes
Decoupled model input from capture size	Model sees 416px regardless of camera size	Yes
Threshold before NMS	NMS sees <30 boxes, not 3549	Yes
No normalisation in preprocessing	Removes two full-frame passes	Yes
Capping OpenCV threads	Measured -4.3%: hypothesis was wrong	No — rejected

2. Results and Performance Analysis

All figures below were produced by scripts in the repository on an Intel Core i5-1135G7 (4 cores, 8 GB RAM, Windows 11), CPU only, after a thermal settling period. Nothing is estimated.

Frame rate across input resolutions

Input size	FPS mean	FPS median	FPS p95	Inference ms	Meets >15 FPS?
320 x 320	53.7	54.4	66.3	16.3	Yes
416 x 416 (default)	38.0	38.4	46.0	22.9	Yes
512 x 512	27.6	27.9	32.7	31.6	Yes
416 (FP32, for comparison)	19.8	19.8	25.4	47.1	Yes



Latency breakdown per frame

Capture	Preprocess	Inference	Post-process	Render	Total
0.03 ms	1.22 ms	22.85 ms	1.71 ms	0.46 ms	26.26 ms

- Inference is 87% of the frame budget. Every other stage combined is 3.4 ms, which is why optimisation effort went into the model rather than the surrounding pipeline.

Accuracy

Measured on 300 labelled COCO val2017 images. No accuracy figure is claimed for webcam footage, which has no ground truth.

Metric	INT8 (shipped)	FP32	Difference
mAP@0.5	0.5183	0.5326	-0.0143 (-2.7%)
mAP@0.5:0.95	0.3316	0.3568	-0.0252 (-7.1%)
Precision @ conf 0.30	0.673	0.694	-0.021
Recall @ conf 0.30	0.483	0.490	-0.007
F1 @ conf 0.30	0.562	0.574	-0.012

- INT8 costs 2.5 mAP points and buys 1.9x the frame rate — a clearly favourable trade on CPU-only hardware, so it became the default.
- Precision (0.67) exceeds recall (0.48): the detector is conservative. What it reports is usually right, but it misses roughly half of what is labelled at the shipped 0.30 threshold.
- Small objects are the weak point — mAP 0.17 on small versus 0.51 on large.

3. Challenges Faced and Solutions

Thermal throttling made the benchmarks meaningless

- Problem:** Four identical back-to-back runs measured 24.3, 21.0, 20.4 and 19.4 FPS with nothing changed but time — a 20% drift, larger than most of the optimisations being measured.
- Solution:** Added a `--settle` option that loads the CPU to a steady thermal state before any measurement. Differences of a few percent are re-checked with counterbalanced A/B runs so ordering cannot masquerade as an effect.

The released model had a hard-coded input size

- **Problem:** The official YOLOX ONNX export fixes the input at 416x416, which made the required multi-resolution benchmark impossible.
- **Solution:** The network is fully convolutional — its head reshapes target [1, 85, -1]. Only the declared input dimensions needed widening. A script relaxes them and verifies that 320, 416 and 512 produce exactly the anchor counts the stride geometry predicts.

The demo video recorded at the wrong speed

- **Problem:** The first real 45-second take was written as a 48-second file playing at 0.49x, while the FPS counter drawn onto those same frames read 30. The video writer took its frame rate from a rolling average that still included the ~600 ms first frame.
- **Solution:** The rate is now measured from frames after warm-up, the calibration window was widened, and the tool warns when the file's rate and the achieved rate diverge by more than 10%. A regression test guards it.

Quantisation was calibrated on the test set

- **Problem:** INT8 quantisation calibrates on real images. The first build used the first 64 images of the evaluation set — the model was tuned on data it was then scored against.
- **Solution:** A selection script now picks 24 permissively-licensed images from outside the evaluation range and commits them, so the build is reproducible and offline. Measured bias was 0.0004 mAP — negligible, but only knowable after checking.

A predicted optimisation turned out to be wrong

- **Problem:** The design document predicted that capping OpenCV's threads would help, since OpenCV and ONNX Runtime both grab every core.
- **Solution:** Counterbalanced A/B testing showed capping cost 4.3%. The default was changed to match the measurement, and the rejected hypothesis is documented rather than quietly deleted.

Live frame rate is limited by the camera, not the code

- **Problem:** After INT8 doubled inference speed, the live frame rate fell from 23 to 15 FPS.
- **Diagnosis:** Measuring the camera alone, with no inference running, showed it delivering exactly 15.0 FPS — auto-exposure had halved its output as the room darkened. The pipeline is now camera-bound, so benchmarks use a deterministic file source and live figures are reported separately.

4. Future Improvements

Improvement	Expected gain	Status
OpenVINO execution provider — targets the idle Intel Iris Xe iGPU alongside the CPU	1.3–2x on top of INT8	Projected, not verified
Move rendering and video encoding to a third thread, overlapping them with the next frame's inference	2–9%	Projected
Object tracking (e.g. ByteTrack) to carry identities between frames and smooth flicker	Better perceived stability, not raw FPS	Not started
Evaluate on the full 5000-image val2017 set rather than a 300-image subset	Tighter per-class figures	Script already supports it
Measure accuracy at 320 and 512 to complete the speed/accuracy curve	Fills the two gaps in the trade-off table	Tooling ready

Known limitations

- Small and distant objects: mAP 0.17 versus 0.51 for large objects.

- Motion blur and low light degrade detection; neither is quantified, because doing so honestly would require a labelled dataset for each.
- Overlapping instances of the same class can be merged by NMS at IoU 0.45.
- Only Windows x86_64 was verified in practice. Linux, macOS and ARM64 wheels all exist, but were not run.
- The INT8 model is built locally and therefore carries no checksum, unlike the FP32 weights which are SHA-256 verified on download.

5. Deliverables and How to Verify Them

Every item below is in the public repository at github.com/Pratik7595/real-time-object-detection.

Deliverable	Location	Verify with
Source code with documentation	src/, README.md	<code>python -m src.main --source 0</code>
Dependency specification	requirements.txt (+ -dev)	<code>pip install -r requirements.txt</code>
Setup and running instructions	README.md, SUBMISSION.md	Six commands, ~3 minutes
Sample input	assets/sample.jpg	640x480, CC BY 2.0
Sample output (visual)	results/sample_detection.png	<code>python scripts/make_sample_output.py</code>
Sample output (data)	results/sample_output.json	9 detections, boxes and scores
Demo video, 43 seconds	Releases tab, tag v1.0	demo.mp4, 4.3 MB
Performance analysis	docs/PERFORMANCE_ANALYSIS.md	<code>python -m src.benchmark</code>
Accuracy metrics	results/accuracy_*.csv	<code>python -m src.evaluate</code>
This report	docs/PROJECT_REPORT.pdf	—
Automated tests	tests/	<code>python -m pytest (68 pass)</code>

Every number quoted in this report traces to a committed file under results/, produced by a script in the repository that can be re-run. Where a figure was not measured, it is labelled as not measured rather than estimated.

Further detail: docs/PERFORMANCE_ANALYSIS.md (full methodology and per-class results), docs/PRD.md (design decisions, including the predictions that measurement later contradicted), docs/DEMO_SHOTLIST.md (how the video was produced).